STEP 1 — Environment Setup

```
# Import required libraries
import numpy as np
import pandas as pd
import re
import string

# Sklearn libraries
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns
```

STEP 2 — Data Loading

```
# Load dataset (update filename if needed)
df = pd.read_csv("/content/SMSSpamCollection", encoding='latin-1')

# Rename columns if needed
df.columns = ["label", "text"]

# Display first 5 rows
df.head()
```

	label	text
0	ham	Ok lar... Joking wif u oni...
1	spam	Free entry in 2 a wkly comp to win FA Cup fina...
2	ham	U dun say so early hor... U c already then say...
3	ham	Nah I don't think he goes to usf, he lives aro...
4	spam	FreeMsg Hey there darling it's been 3 week's n...

Next steps: Generate code with df New interactive sheet

Check class distribution:

```
df["label"].value_counts()
```

label	count
ham	4824
spam	747

dtype: int64

STEP 3 — Text Preprocessing

```
def preprocess_text(text):
    text = text.lower()  # lowercase
    text = re.sub(r'[^\w\s]', '', text)  # remove punctuation
    return text

df["clean_text"] = df["text"].apply(preprocess_text)

df.head()
```

	label	text	clean_text
0	ham	Ok lar... Joking wif u oni...	ok lar joking wif u oni
1	spam	Free entry in 2 a wkly comp to win FA Cup fina...	free entry in 2 a wkly comp to win fa cup fina...
2	ham	U dun say so early hor... U c already then say...	u dun say so early hor u c already then say
3	ham	Nah I don't think he goes to usf, he lives aro...	nah i dont think he goes to usf he lives aroun...
4	spam	FreeMsg Hey there darling it's been 3 week's n...	freemsg hey there darling its been 3 weeks now...

Next steps: Generate code with df New interactive sheet

STEP 4 — Feature Extraction

```
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df["clean_text"])
```

Convert labels to numeric:

```
y = df["label"].map({"ham": 0, "spam": 1})
```

STEP 5 — Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

STEP 6 — Model Implementation

```
model = MultinomialNB()
model.fit(X_train, y_train)
```

```
▾ MultinomialNB
MultinomialNB()
```

STEP 7 — Model Evaluation

```
y_pred = model.predict(X_test)
```

Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.979372197309417
```

Precision, Recall, F1

```
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

```
Precision: 0.9254658385093167
Recall: 0.93125
F1 Score: 0.9283489096573209
```

Classification Report

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       955
```

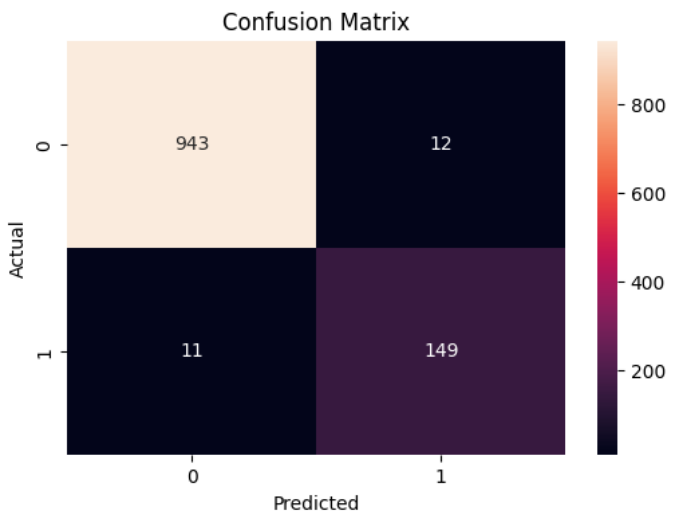
```
           1       0.93      0.93      0.93       160

    accuracy                           0.98      1115
   macro avg       0.96      0.96      0.96      1115
weighted avg       0.98      0.98      0.98      1115
```

Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt="d")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



STEP 8 — Result Analysis (Write in Report)

Example:

The Naive Bayes model achieved high accuracy (~95–98% for SMS dataset).

Precision indicates how many predicted spam messages were actually spam.

Recall shows how many spam messages were correctly detected.

High F1-score indicates balanced performance.

Naive Bayes works well because text data is high-dimensional.

It assumes feature independence, which is not always realistic.

It is fast and computationally efficient.

However, it may fail for complex language patterns.

It does not capture context like deep learning models.

Overall, it is a strong baseline model for text classification.